README

Helix OTA — Future Phase: Microsoft Windows Support

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	future (research outline)
Status summary	Basic planning for extending Helix OTA to Windows (10/11, IoT/LTSC, Server) via the OS-adapter seam. Control plane unchanged; a Windows device adapter is added. Multiple Windows update/packaging standards must all be supported.
Issues	Windows has several coexisting mechanisms (MSIX, MSI, WinGet, Windows Update Agent/WUfB, IoT image servicing); the adapter must cover the relevant set per target.
Issues summary	Heterogeneous Windows servicing surface; needs per-class adapters.
Fixed	initial future outline
Fixed summary	Captured candidate mechanisms + decision axes per the “support it all” mandate.
Continuation	Promote to a numbered phase (e.g. 3.0.0-windows) with full specs + a spike per mechanism after Linux support lands.

Table of contents

§1. Scope

Windows 10/11 desktop, Windows IoT Enterprise (closest analogue to the appliance use-case), and Windows Server. The Helix control plane is reused; a Windows **device adapter** (Go service or .NET) implements the OS-adapter interface + `ota-protocol`.

§2. Candidate mechanisms

- **MSIX** — modern packaged-app format with atomic install/uninstall + automatic rollback on failed install; closest to the safe-update model; App Installer supports auto-update from a URL.
- **MSI** — legacy installer; transactional but heavier; needed for non-packaged payloads.
- **WinGet** — package manager + manifest model usable for fleet distribution of app payloads.
- **Windows Update Agent / Windows Update for Business / WUfB-DS** — OS-level servicing; relevant for full-OS updates and IoT image servicing.
- **IoT image servicing (Device Update for IoT Hub concepts)** — A/B-style image updates for Windows IoT; the closest match to the appliance OTA model.

§3. OS-adapter mapping

The same universal seam (`CheckForUpdate/Download/Verify/Install/Rollback/GetCapabilities`). Each mechanism becomes a concrete adapter; target class selects it (appliance/IoT → image servicing or MSIX; desktop app payloads → MSIX/WinGet; OS patching → WU/WUfB).

§4. Security & integrity

Authenticode signing + the Helix detached signature + SHA-256, verified by the control plane on upload and by the device adapter before install. TLS 1.3 transport (HTTP/3→HTTP/2). The trust model (TUF/Uptane seam) carries over from the Android/Linux phases — the trust layer is OS-agnostic by design.

§5. Open spikes

Per mechanism: produce a signed package/image → serve from the Go control plane → drive install via the chosen API → confirm rollback/uninstall semantics. Confirm which mechanisms expose a reliable post-install health signal for the health-gated halt.

---

§6. addition_3_research_routing (rev 2 — 2026-06-08)

This section folds addition-#3's Windows research into this canonical phase dir per synthesis §11. It **extends** the original outline above; nothing above is removed. The source doc self-numbered `1.2.0-windows-support`; the canonical destination is **`1.X-windows`** (promoted to a concrete version after Linux support lands), per synthesis §11's `1.X-windows/` routing.

§6.1 source_research

Source: `additions/initial_research_03/helix_ota_big/1.2.0-windows-support/docs/WINDOWS_OTA_RESEARCH.md`. Cited sections, summarized (NOT copied):

- **§1 Windows Update Architecture** — WUA API (§1.2), WSUS (§1.3), Windows Update for Business (§1.4), Intune/MECM packaging (§1.5), third-party mechanisms (§1.6). Establishes the heterogeneous servicing surface already noted in §2 above.
- **§2 Windows Service Client Design** — OTA client as a Windows Service, a **Go implementation as a Windows service** (§2.2), BITS for downloads (§2.3), Event Log integration (§2.4), tray notifications (§2.5), auto-start/recovery (§2.6).
- **§3 MSI/MSIX Package Distribution** — MSI creation (§3.1), **MSIX** modern packaging with atomic install/rollback (§3.2), WinGet repo integration (§3.3), a custom update-catalog format (§3.4), **Authenticode** signing (§3.5).
- **§4 Windows-Specific Security** — code-signing requirements (§4.1), SmartScreen (§4.2), UAC handling (§4.3), **Secure Boot + TPM** (§4.4), Windows credential/cert store (§4.5).
- **§5 Windows Adapter Implementation** — the `OSAdapter` interface for Windows (§5.1) and registry-based configuration (§5.2) — the concrete form of the universality seam in §3 above.

§6.2 reconciliation_to_locked_decisions

- **Control plane unchanged:** Go + **Gin**, REST primary (HTTP/3→HTTP/2, Brotli). A Windows device adapter (Go Windows service preferred per source §2.2, .NET as alternative) implements the OS-adapter interface; the source's Go-service path is the catalogue-aligned choice.
- **Vocabulary:** canonical **releases + deployments**; re-base `updates`/`rollouts`.
- **Signing/trust: ed25519 + SHA-256** Helix detached signature is the canonical gate; **Authenticode** is an *additional* platform-native layer (required for SmartScreen/UAC trust), not a replacement. JWT bearer device auth; TPM/Secure-Boot is hardening, parallel to the TUF/Uptane seam.
- **Module path / submodules:** re-base any `helix-*` names to `ota-*` under `github.com/HelixDevelopment`; PUBLIC repo creation gated on G11 verification; reuse the verified catalogue.
- **Coverage:** ≥90% floor on safety-critical adapter paths (verify, install, rollback/uninstall).

§6.3 what_must_be_specced_before_this_phase_starts

1. ADR to pick the per-class mechanism (appliance/IoT image servicing or MSIX; desktop app payloads → MSIX/WinGet; OS patching → WU/WUfB).
2. Decide client host language (Go Windows service vs .NET) and freeze it against the OS-adapter interface.
3. Spike each mechanism end-to-end (signed package/image → serve from Go control plane → install via the API → confirm rollback/uninstall) and confirm a **post-install health signal** for the health-gated halt.
4. Re-based submodule + PUBLIC repo verification; Authenticode signing pipeline spec layered over the ed25519 gate.
5. Promote `1.X-windows` to a concrete version once Linux support lands.

§6.4 anti_bluff_unverified_register

Per Constitution §11.4.6 / §7.1 — MUST NOT propagate as fact:

- WUA/BITS/MSIX/WinGet/WUfB capability and rollback-semantics claims — UNVERIFIED until spiked on real Windows targets.
- TPM/Secure-Boot integration behavior and SmartScreen/UAC trust flows — UNVERIFIED.
- Which mechanisms expose a reliable post-install health signal — UNVERIFIED (open question from §5 above).
- Source-doc Go-Windows-service / `OSAdapter` reference + any `helix-*` names — reference only; non-canonical until re-based.
- All cited HelixConstitution §11.4.x clause numbers remain UNVERIFIED except the six confirmed in `tests/test_strategy.md` §13.